Git & VS Code: Der komplette Workflow

Ein praktisches Beispiel für das Repository: CuD_Aufgabe_012

> **Voraussetzungen:** Bevor du startest, stelle sicher, dass Git auf deinem System installiert ist und du in Visual Studio Code (VS Code) mit deinem GitHub-Konto angemeldet bist. Die folgenden Schritte zeigen dir den gesamten Lebenszyklus einer Code-Änderung.

1 Repository klonen (Clone)

Der allererste Schritt ist es, eine exakte Kopie des Repositories vom GitHub-Server auf deinen lokalen Computer herunterzuladen. Dieser Vorgang nennt sich "Clonen" und muss für jedes Projekt nur ein einziges Mal durchgeführt werden.

Die URL deines Repositories: `https://github.com/dk3rn/CuD_Aufgabe_012.git`

WEG A: ÜBER DIE VS CODE OBERFLÄCHE

1. Öffne VS Code und drücke `Strg + Shift + P` (Befehlspalette).
2. Tippe **Git: Clone** ein und drücke Enter.
3. Füge die URL deines Repositories in die Leiste oben ein und bestätige mit Enter.
4. Wähle einen lokalen Ordner (z.B. deine Dokumente) als Speicherort.
5. Klicke im Popup unten rechts auf **Öffnen**, um das geklonte Projekt zu laden.

WEG B: ÜBER DAS TERMINAL

1. Öffne das Terminal in VS Code (`Strg + ö`).
2. Navigiere mit `cd` in deinen gewünschten Zielordner.
3. Gib folgenden Befehl ein:

```
git clone
https://github.com/dk3rn/CuD_Aufgabe_012.git
```

Öffne den neuen Ordner danach über `Datei` > `Ordner öffnen...`

> **Tipp:** Wenn das Repository privat ist, wirst du beim ersten Klonen möglicherweise aufgefordert, dich über den Browser bei GitHub anzumelden und VS Code die entsprechenden Rechte zu gewähren.

2 Dateien ändern und speichern

Nachdem du das Projekt geöffnet hast, kannst du beginnen, an deinen Aufgaben zu arbeiten. In der linken Seitenleiste (dem Explorer) siehst du alle Dateien des Projekts.

1. Öffne eine bestehende Datei (z.B. `index.html` oder `main.py`) oder erstelle eine neue Datei für die Aufgabe 012.
2. Nimm deine Änderungen im Code-Editor vor.
3. Speichere die Datei mit `Strg + S`.

Sobald eine Datei geändert und gespeichert wurde, bemerkt Git dies. In der linken Aktivitätenleiste siehst du nun eine kleine Zahl am "Quellcodeverwaltung"-Symbol (Source Control). Dies zeigt an, wie viele Dateien verändert wurden.

3 Änderungen vormerken (Add / Stage)

Bevor du eine Änderung in der Versionsgeschichte speichern kannst, musst du Git genau mitteilen, *welche* der geänderten Dateien für den nächsten Speicherpunkt berücksichtigt werden sollen. Dieser Zwischenbereich wird "Staging Area" genannt.

WEG A: ÜBER DIE VS CODE OBERFLÄCHE

1. Klicke in der ganz linken Symbolleiste auf das **Quellcodeverwaltung**-Icon (Verzweigungs-Symbol) oder drücke `Strg + Shift + G`.
2. Unter dem Bereich "Änderungen" siehst du deine geänderten Dateien.
3. Fahre mit der Maus über die Datei, die du hinzufügen möchtest, und klicke auf das **Plus-Symbol (+)** daneben (Änderungen mergen / Stage Changes).
4. Die Datei verschiebt sich nun in den Bereich "Gestagte Änderungen" (Staged Changes).

WEG B: ÜBER DAS TERMINAL

Im Terminal nutzt du den Befehl `git add`.

Um eine **bestimmte Datei** hinzuzufügen:

```
git add dateiname.ext
```

Um **alle** geänderten Dateien auf einmal hinzuzufügen (sehr häufig genutzt):

```
git add .
```

Der Punkt `.` steht hierbei für das gesamte aktuelle Verzeichnis.

4 Änderungen lokal speichern (Commit)

Nun, da sich die Dateien in der Staging Area befinden, erstellst du einen Commit. Ein Commit ist wie ein "Foto" oder ein permanenter Speicherpunkt deines Codes zu diesem Zeitpunkt. Jeder Commit benötigt eine kurze, beschreibende Nachricht.

WEG A: ÜBER DIE VS CODE OBERFLÄCHE

1. Bleibe in der Quellcodeverwaltung-Ansicht.
2. Tippe oben in das Textfeld **"Nachricht"** eine kurze Beschreibung ein, z.B. *"Aufgabe 12 gelöst"* oder *"Tippfehler in README korrigiert"*.
3. Klicke auf den blauen Button **Übernehmen (Commit)**.

WEG B: ÜBER DAS TERMINAL

Verwende den `commit` Befehl zusammen mit dem Parameter `-m` (für Message), um die Nachricht direkt anzuhängen:

```
git commit -m "Aufgabe 12 gelöst"
```

Achte darauf, die Nachricht in Anführungszeichen zu setzen.

5 Auf den Server laden (Push)

Der Commit existiert bisher nur lokal auf deinem eigenen Computer. Damit dein Dozent oder dein Team die Änderungen auf GitHub (`https://github.com/dk3rn/CuD_Aufgabe_012.git`) sehen kann, musst du sie auf den Server hochladen (Pushen).

WEG A: ÜBER DIE VS CODE OBERFLÄCHE

1. Nachdem du den Commit erstellt hast, verwandelt sich der Commit-Button oft in einen Button namens **Änderungen synchronisieren**. Klicke darauf.
2. Alternativ: Klicke auf die **drei Punkte (...)** oben rechts in der Quellcodeverwaltung und wähle **Push** (Senden) aus dem Menü.

WEG B: ÜBER DAS TERMINAL

Um deine lokalen Commits auf den Server (origin) in deinen Hauptbranch (meist `main`) hochzuladen, nutze:

```
git push origin main
```

Wenn dein Branch anders heißt (z.B. `master`), passe das letzte Wort entsprechend an.

Zusammenfassung der Terminal-Befehle (Der tägliche Rhythmus):

`git add .` ➔ `git commit -m "Deine Nachricht"` ➔ `git push`

Sobald du diesen Ablauf ein paar Mal gemacht hast, wird er zur Routine bei jeder neuen Code-Änderung.